

1 Key Concepts

1.1 Divide and Conquer

The divide-and-conquer paradigm captures a large family of algorithms, for a wide variety of computational problems, that all follow a common template. A divide-and-conquer algorithm for a problem, given an input, *divides* it into smaller input instances of the same problem. It then solves these instances *recursively*, and finally *combines* the solutions in some appropriate way to get a solution for the original input.

Key ingredients. The following ingredients will be present in any divide-and-conquer algorithm. Designing them in the “right” way can often be non-obvious, and requires creative thinking.

1. *Base case.* When the input is small enough, the algorithm just directly outputs an answer. (The base case is often when the input size is zero or one, like sorting an empty or one-element array.)
2. *Divide.* The input is subdivided into some subinputs (which are typically smaller by some constant factor) and the algorithm is called recursively on them. (For example, when the input is an array, it may be split into a “left half” and a “right half.” When the input is a tree, it may be divided into its left and right subtrees.)
3. *Conquer.* The outputs of the recursive calls (on the subinputs) are combined, perhaps together with some processing of the original input, to compute a final output for the full input.

Proving correctness. Proving the correctness of a divide-and-conquer algorithm usually closely mirrors a proof by induction. The proof is structured in two parts:

1. *Base case.* A (usually trivial) proof that the base-case output is correct.
2. *Recursive/inductive step.* A proof that *if* the outputs of the recursive calls are correct, *then* the final output (for the original input) is also correct. This is structured very much like the inductive step of an induction proof: first *assume* that the recursive calls return correct answers, which is known as the *inductive/recursive hypothesis*. Then use this assumption, along with the work performed by the conquer step (and some structural properties of the problem), to show that the final output is correct (for the original input).

By these two steps, it follows that the algorithm is correct on *any* input, of any size. (Formally, this conclusion is reached by *strong induction*; see the discussion in [Handout 2](#) for details.)

Runtime analysis. To analyze the running time $T(n)$ of a divide-and-conquer algorithm, we first derive a *recurrence relation* for it. This is typically an equation that expresses $T(n)$ in terms of values of T for smaller input sizes (representing the running time of the recursive calls), plus some additional term (representing the running time of the conquer step). Often, such a recurrence relation can then be “solved” by invoking the Master Theorem ([Theorem 1](#)), which gives an asymptotic “closed form” for $T(n)$ that is not self-referential.

Many divide-and-conquer algorithms have runtime recurrence relations of the form

$$T(n) = k \cdot T(n/b) + O(n^d),$$

where k is the number of recursive calls the algorithm makes on subinputs, n/b is the size of each subinput, and $O(n^d)$ is the runtime of the conquer step.

For example, MERGESORT has a runtime recurrence of $T(n) = 2T(n/2) + O(n)$, where n represents the number of elements to be sorted. This is because it makes two recursive calls on $n/2$ elements each, then does an $O(n)$ -time MERGE step to interleave (or “merge”) the two sorted results. By the Master Theorem, this has a closed form of $T(n) = O(n \log n)$.

1.2 Master Theorem

The Master Theorem provides “cookbook solutions” for recurrence relations of a certain form. In particular, it provides asymptotic bounds for recurrences that describe the runtime of many divide-and-conquer algorithms you have seen and will see in the future (e.g., Mergesort, Karatsuba multiplication). However, there are also some divide-and-conquer algorithms for which the Master Theorem is not applicable.

Theorem 1 (The Master Theorem). *Let $T(n) = k \cdot T(n/b) + O(n^d)$ for constants $k \geq 1, b > 1, d \geq 0$. Then*

$$T(n) = \begin{cases} O(n^d) & \text{if } k < b^d \\ O(n^d \log n) & \text{if } k = b^d \\ O(n^{\log_b k}) & \text{if } k > b^d. \end{cases}$$

Moreover, if the additive term of the recurrence is a big- Θ expression (rather than a big- O one), then all the closed-form solutions for $T(n)$ are big- Θ bounds as well.

The Master Theorem also generalizes to cases where the additive term (representing the “combine” runtime) has logarithmic factors.

Theorem 2 (The Master Theorem with Log Factors). *Let $T(n) = k \cdot T(n/b) + \Theta(n^d \log^w n)$ for constants $k \geq 1, b > 1, d \geq 0, w \geq 0$. Then*

$$T(n) = \begin{cases} \Theta(n^d \log^w n) & \text{if } k < b^d \\ \Theta(n^d \log^{w+1} n) & \text{if } k = b^d \\ \Theta(n^{\log_b k}) & \text{if } k > b^d. \end{cases}$$

This is a generalization of the Master Theorem in that if $w = 0$, then we obtain the ordinary Master Theorem ([Theorem 1](#)).

2 Discussion Exercises

1. Majority Element

Given an array $A[1, \dots, n]$ of n elements, the *majority element* of A is the element that appears strictly more than $n/2$ times, if such an element exists. Note that if a majority element exists, then it is unique—so we say “*the* majority element” instead of “*a* majority element”. The algorithm MAJORITYELEMENT below outputs the majority element of A if it exists, otherwise it outputs a special symbol $\perp$. For simplicity we assume that n is a power of two, so that we can always split the array exactly in half (and don’t have to deal with annoying floors and ceilings).

```
1: function MAJORITYELEMENT( $A[1, \dots, n]$ )
2:   if  $n = 1$  then return  $A[1]$ 
3:    $x \leftarrow$  MAJORITYELEMENT( $A[1, \dots, n/2]$ )
4:    $y \leftarrow$  MAJORITYELEMENT( $A[n/2 + 1, \dots, n]$ )
5:   if  $x \neq \perp$  then
6:     iterate over  $A$ , counting the number of occurrences of  $x$ 
7:     if the number of occurrences of  $x$  in  $A$  is  $> n/2$  then return  $x$ 
8:   if  $y \neq \perp$  then
9:     iterate over  $A$ , counting the number of occurrences of  $y$ 
10:    if the number of occurrences of  $y$  in  $A$  is  $> n/2$  then return  $y$ 
11:  return  $\perp$ 
```

- (a) Give a recurrence, and asymptotic closed-form solution, for the running time of MAJORITYELEMENT.

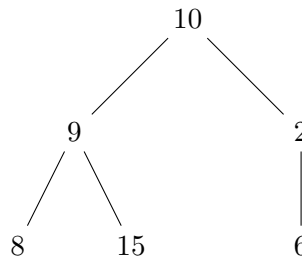
- (b) Show the correctness of the MAJORITYELEMENT algorithm by first proving the following statement:

If z is the majority element of an array $A[1, \dots, n]$, then z must be the majority element of at least one of the subarrays $A[1, \dots, n/2]$ and $A[n/2 + 1, \dots, n]$.

2. Complete Binary Tree Local Max

A *complete* binary tree is a binary tree in which every level, except possibly the last (leaf) level, is completely filled, and all the nodes in the last level are as far left as possible.

Consider a complete binary tree $T = (V, E, r)$ rooted at r where each vertex is labelled with an integer. A vertex $v \in V$ is called a *local maximum* if the label of v is greater than or equal to the labels of all its adjacent nodes—namely, its parent and children (whichever ones exist).



In this example, the local maxima are 10, 15, and 6.

Suppose you are given such a tree where the labelling is given implicitly, i.e., the only way to determine the label of the vertex v is to visit v and query the label. Give an algorithm that computes a local maximum of T using just $O(\log|V|)$ vertex-label queries. Note: T may contain multiple local maxima; the algorithm just needs to return one of them.

3. Divide and Conquer for Integer Multiplication

Karatsuba's algorithm is a fast multiplication algorithm that computes the product of two positive integers x and y using three multiplications of smaller numbers, each with about half as many digits as x and y , plus some additions, subtractions, and digit shifts. In this question, we build on the idea by dividing the numbers into smaller pieces. Namely, we break each argument x, y into *three* equal-sized pieces, of one-third the size. (For simplicity, assume that the size of x and y is a power of three, which can be achieved by "padding" them with some leading zeros; this does not affect the asymptotics.)

Similar to Karatsuba's algorithm, a naive divide-and-conquer algorithm, which we call $\mathcal{A}$, can multiply x and y using **9 recursive multiplications** of numbers that are $n/3$ digits long, along with some linear-time work to combine the results.

Using some clever algebraic manipulations, it is possible to devise a "super-Karatsuba" algorithm that uses only **5 recursive multiplications** of numbers that are about $n/3$ digits long! As before, the results can still be combined in linear time. We call this Algorithm $\mathcal{B}$.

For each algorithm $\mathcal{A}$ and $\mathcal{B}$, derive the recurrence relation for the time complexity $T(n)$, then find an asymptotic closed-form solution for $T(n)$. Which of the algorithms $\mathcal{A}$ and $\mathcal{B}$ is asymptotically faster? How do they compare (asymptotically) to Karatsuba's Algorithm?

Optional content: For those interested, we give the key equations underlying algorithms $\mathcal{A}$ and $\mathcal{B}$ here:

If x, y are n -bit integers, broken up as

$$\begin{aligned}x &= a \cdot 2^{2n/3} + b \cdot 2^{n/3} + c \\y &= d \cdot 2^{2n/3} + e \cdot 2^{n/3} + f\end{aligned}$$

then

$$xy = ad \cdot 2^{4n/3} + (ae + bd)2^n + (af + be + cd)2^{2n/3} + (bf + ce)2^{n/3} + cf = A2^{4n/3} + B2^n + C2^{2n/3} + D2^{n/3} + E,$$

where

$$\begin{aligned}A &= ad \\B &= ae + bd \\C &= af + be + cd \\D &= bf + ce \\E &= cf.\end{aligned}$$

In Algorithm $\mathcal{A}$, computing each value of $ad, ae, af, bd, be, bf, cd, ce, cf$ can be regarded as a sub-problem. We first solve the sub-problems and then compute xy by merging the solutions of sub-problems to get A, B, C, D, E based on the equations above. In Algorithm $\mathcal{B}$, we compute A, B, C, D, E by computing X_0, X_1, X_2, X_3, X_4 below, where as usual the multiplication in each of them is computed recursively:

$$\left\{ \begin{array}{l} X_0 = cf \\ X_1 = (a + b + c)(d + e + f) \\ X_2 = (4a + 2b + c)(4d + 2e + f) \\ X_3 = (a - b + c)(d - e + f) \\ X_4 = (4a - 2b + c)(4d - 2e + f) \end{array} \right. \implies \left\{ \begin{array}{l} A = \frac{1}{24}(6X_0 - 4X_1 + X_2 - 4X_3 + X_4) \\ B = \frac{1}{12}(-2X_1 + X_2 + 2X_3 - X_4) \\ C = \frac{1}{24}(-30X_0 + 16X_1 - X_2 + 16X_3 - X_4) \\ D = \frac{1}{12}(8X_1 - X_2 - 8X_3 + X_4) \\ E = X_0 \end{array} \right. .$$

4. 2D Closest-Pair algorithm

This problem is a review of some key parts of the analysis of the 2D Closest-Pair algorithm from class; it does not contain any new content. The analysis of the algorithm considers a 2δ -by- δ rectangular strip, split by the median line into two δ -by- δ square areas. All points that the algorithm compares in the “left” subset (which lie to the left of, or on, the median line) are at distance at least δ from each other, and similarly, all points in the “right” subset (which must lie to the right of, or on, the median line) are at distance at least δ from each other.

In particular, to show that the algorithm performs better (asymptotically) than the naïve algorithm, we show that the “conquer” portion of this algorithm needs to do only a *constant* number of comparisons per point p . The key fact is that any other point on the “other side,” and within distance δ of p , must be in the bounding rectangle, and only a constant number of points can “fit” within that rectangle.

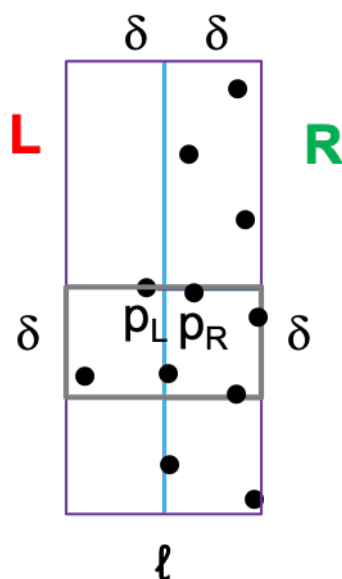


Figure 1: The 2δ -by- δ rectangle (indicated by the gray rectangle) used in the analysis and points within δ of the median line ℓ .

- (a) Prove that the left square area can contain at most 4 points from the left subset, and similarly that the right square area can contain at most 4 points from the right subset.

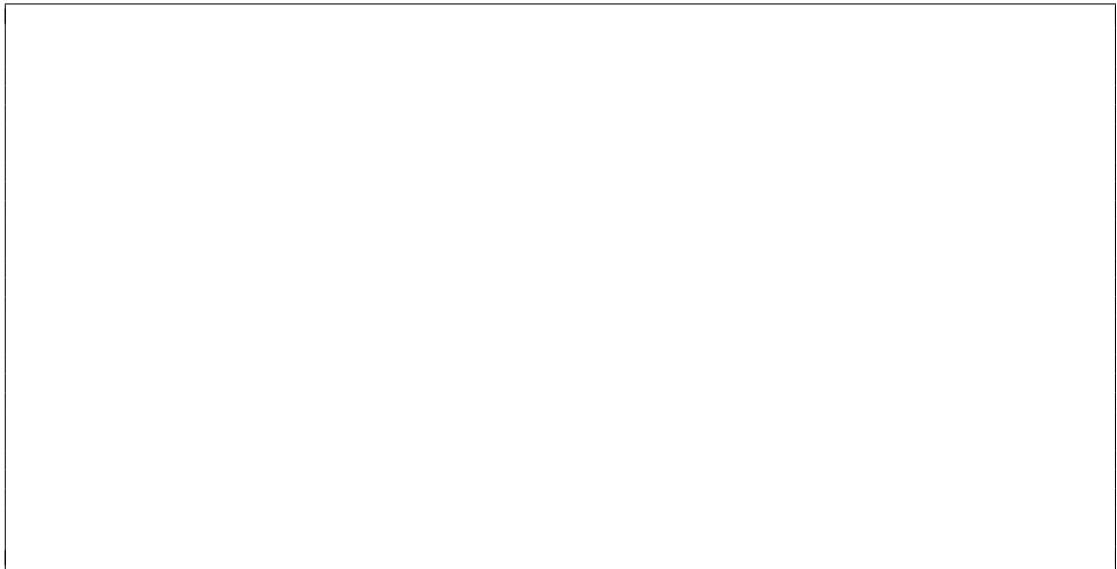
Hint: partition each square area into four congruent square sub-areas, and show that each sub-area can have at most one point from the relevant subset.

Hint: The pigeonhole principle may be useful (but not necessary) for your proof. It states that if n items are placed in m boxes and $n > m$, then at least one box contains more than one item.



- (b) Use the previous part to argue why it suffices for the “conquer” part of the Closest-Pair algorithm to check, for each point in the y -sorted array of points within δ of the median line, only the subsequent 8 points in the array.

(Note: the value 8 here is not optimal, but for the asymptotic conclusion we just need some constant bound.)



3 Additional Problems

1. SlowSort

Consider the sorting algorithm *slowsort*, which can be represented with the following pseudocode. What is the most precise recurrence relation for the running time? Does the Master Theorem apply, and if so, what closed-form solution does it give?

```

1: function SLOWSORT( $A[1, 2, \dots, n]$ ) // n is length of A
2:   SLOWSORT( $A[1, \dots, \lfloor \frac{n}{2} \rfloor]$ ) // sort both halves of the array recursively
3:   SLOWSORT( $A[\lfloor \frac{n}{2} \rfloor + 1, \dots, n]$ )
4:   if  $A[\lfloor \frac{n}{2} \rfloor] > A[n]$  then // largest item in first half is greater than largest in the second
5:     swap  $A[\lfloor \frac{n}{2} \rfloor]$  and  $A[n]$  // put largest item in the unsorted array at the end
6:   SLOWSORT( $A[1, \dots, n - 1]$ ) // recursively sort the entire array minus one element
7:   return
```

- ☐ $T(n) = 2T\left(\frac{n}{2}\right) + T(n - 1) + O(1)$; the Master Theorem gives $T(n) = O(n^2 \log n)$
- ☐ $T(n) = 2T\left(\frac{n}{2}\right) + T(n - 1) + O(1)$; the Master Theorem does not apply
- ☐ $T(n) = 3T\left(\frac{n}{2}\right) + O(1)$; the Master Theorem is unusable
- ☐ $T(n) = 3T\left(\frac{n}{2}\right) + O(1)$; the Master Theorem gives $T(n) = O(n^{\log_2 3})$
- ☐ This cannot be written as a relation; the Master Theorem is unusable

2. Pikachu: I trade you!

You are a savvy investor who is interested in an especially hot asset that is now on the market: a rare Pikachu Pokémon trading card. You want to purchase and sell this card *just once*, and time these events to maximize your rate of return, i.e., the percentage by which the card's value increases between purchase and sale.

Your top trading-desk researchers have prepared their best estimates of how much the card's value will change on each of the next $n \geq 2$ days. These estimates are given as nonnegative

real *multiplicative* factors, i.e., a 10% gain is represented by 1.1, and a 20% loss is represented by 0.8. Given the array $A[1, \dots, n]$ of these estimates, you want to find indices $1 \leq i \leq j \leq n$ that *maximize*

$$A[i] \cdot A[i+1] \cdots A[j] ,$$

i.e., the aggregate growth in value if the card is purchased at the *start* of day i and sold at the *end* of day j . Assume that at least one day will have a growth factor greater than 1 (otherwise there would be no reason to buy and sell the card, and you're too savvy for that).

For instance, if $A[1, \dots, 6] = [0.8, 1.1, 0.95, 1.5, 0.8, 1.25]$, then the maximum aggregate growth is 1.5675, which is achieved by $i = 2, j = 6$, and also by $i = 2, j = 4$. So, an optimal choice of i and j is not necessarily unique, but the maximum aggregate growth is unique.

In this question you will design a divide-and-conquer algorithm that solves this problem. You may assume that two real numbers can be multiplied in constant time, and for simplicity, you may assume that n is a power of two (if you wish).

- (a) Suppose that for some day k , where $1 \leq k < n$, you want to optimize the aggregate growth under the constraint that you own the card on day k and sell it strictly after day k —i.e., you must purchase it at the start of some day $i \leq k$ and must sell it at the end of some day $j > k$.

Give an algorithm that solves this problem in $O(n)$ time. On input $A[1, \dots, n]$ and k , it should output the maximum aggregate growth (which might be less than 1 under these constraints), and some corresponding purchase and sale days i, j that achieve it.

- (b) Now, give a divide-and-conquer algorithm that solves the main problem in $O(n \log n)$ time. Given the array $A[1, \dots, n]$, it should output the optimal aggregate growth and some corresponding purchase and sale days i, j that achieve it.